DALHOUSIE UNIVERSITY

Performance Estimation

Holdout, random subsampling, bootstrap, and cross-validation

Prof. Gabriel Spadon, PhD

Faculty of Computer Science

spadon@dal.ca

1 Resubstitution estimates, and why they are unreliable

Performance estimation decides which rows a score may be computed on, how many times the computation is repeated, and what uncertainty goes with the answer.

Performance estimation has two goals. The first is to help the analyst select the best prediction model from a set of alternatives. The second is to give end-users a reliable estimate of how the chosen model will perform on future, unseen data. Selection compares candidates. Reporting makes a claim to someone outside the project. Section 9 keeps the two apart.

Every method in this handout obeys one rule.

The Golden Rule

Data used for evaluating (or comparing) any models should never be seen or used during model development.

Unless stated otherwise every number comes from `iris` [7] ($N = 150$, three balanced classes of 50) with a 3-nearest-neighbour classifier and seed 3141.

What it is for A resubstitution score is the one you get for free, and it is the one you must not report. Seeing why tells you what every other method in this class is fixing.

Definition A **resubstitution estimate** evaluates a model's predictive performance on the same data used to train it. It is unreliable for two reasons.

1. **The model was fitted to score well on those rows.** Fitting means optimizing performance on the available data, for example minimizing MSE or maximizing accuracy. A good training score is therefore expected, and the resubstitution estimate is **over-optimistic**. The training score is an accurate measurement of how well the model fits its own rows, which is not the quantity anyone wants.
2. **The data set is only a sample.** The available data are a **sample** of the population, the whole group we want to predict for. We want performance on the whole distribution, and we rarely have the population. The best we can do is evaluate the model on *different samples* of that distribution.

Every method below manufactures those different samples out of the one sample we have, by splitting it once, splitting it repeatedly, resampling it with replacement, or partitioning it into folds.

Worked example Fit k-nearest-neighbour classifiers on `iris` for a range of k, with scikit-learn's default settings. Score each one twice, once by resubstitution and once by stratified 10-fold cross-validation (`StratifiedKFold(10, shuffle=True, random_state=3141)`). Table 1 gives the result.

Table 1. Resubstitution and cross-validation error across k

k	1	3	5	7	9	11	15	21	31	51
Resubstitution error	**0.0000**	0.0400	0.0333	0.0267	0.0200	0.0267	0.0133	0.0200	0.0400	0.0467
10-fold CV error	**0.0400**	0.0467	0.0400	0.0333	0.0400	0.0333	0.0333	0.0400	0.0467	0.0733

At $k = 1$ every training row is its own nearest neighbour, so the resubstitution error is exactly zero by construction. The cross-validated error is 0.0400. Selecting k on the resubstitution row picks $k = 1$.

The CV row reaches its minimum, 0.0333, at $k = 7$, 11 and 15, and places $k = 1$ among the worse choices. The optimism is not a constant offset that cancels when models are compared. It is largest where the model is most flexible, so it distorts the comparison itself. Some entries depend on how ties between equally distant neighbours are broken. Forcing `algorithm="brute"` moves the $k = 9$ CV error from 0.0400 to 0.0267.

Code In Python, `cross_validate(..., return_train_score=True)` (the listing in Section 5) prints the resubstitution score next to the held-out score for every fold, so the optimism is visible in the output.

2 Averaging repeated scores

What it is for One score from one test set is one draw. A reliable estimate repeats the evaluation and summarizes the repeats.

What a reliable estimate needs The goal is a reliable estimate of the expected prediction error of a model on the unknown data distribution.

- The estimate is computed on unseen cases, a **test set**.
- The evaluation is repeated, giving several scores computed on different subsamples of the data.
- The average of those scores is the estimate. It is a more robust estimate of the true population mean prediction error than any one score.
- The standard error of the estimate measures its uncertainty.

Formula Given K error estimates $E_1, E_2, \ldots, E_K$, the sample mean prediction error is

$$\bar{E} = \frac{1}{K} \sum_{i=1}^{K} E_i,$$

and, for independent scores, its standard error is

$$SE(\bar{E}) = \frac{s_E}{\sqrt{K}}, \qquad \text{where} \qquad s_E = \sqrt{\frac{1}{K-1} \sum_{i=1}^{K} \left(E_i - \bar{E}\right)^2}.$$

Worked example Stratified 5-fold cross-validation of 3-NN on `iris` produces the five fold errors 0.0000, 0.0333, 0.0000, 0.0667, 0.0667. Then

$$\bar{E} = \frac{0.1667}{5} = 0.0333, \qquad s_E = \sqrt{\frac{0.004444}{4}} = 0.0333, \qquad SE(\bar{E}) = \frac{0.0333}{\sqrt{5}} = 0.0149.$$

Report the mean 0.0333 and the fold standard deviation 0.0333 as descriptions. The value 0.0149 is a *naive* standard error, because it assumes independent scores. Cross-validation training sets overlap, so it is not a valid uncertainty interval for the generalization error [2]. Increasing K or repeating the folds does not create independent observations.

3 Holdout and random subsampling

What it is for Holdout is the simplest way to obey the Golden Rule. Random subsampling repeats it so that the variability across splits can be seen.

Holdout procedure

1. Split the data into a training set and a test set. A common split is 70 per cent training, 30 per cent testing.
2. Train a model using the training set.
3. Make predictions for the test set and calculate the performance score.

The split is made before anything is fitted, and the test rows are not looked at again.

What one split cannot tell you On a very small data set a test set that is too small gives unreliable estimates, and a training set that is too small gives a worse model than the data could support. The two failures pull in opposite directions, so no split ratio rescues a small data set. At any size, one split cannot measure variability across splits.

Worked example Hold the model fixed at 3-NN on `iris` and vary only the random 70/30 split, 1000 times (seeds 0–999). Table 2 gives the result.

Table 2. Error rate statistics over a thousand random splits

Statistic over 1000 random 70/30 splits	Error rate
Mean	0.0393
Standard deviation	0.0232
Minimum (best split)	0.0000
Maximum (worst split)	0.1333
Central 95 per cent of splits	0.0000 to 0.0889
Seed 42	0.0000
Seed 1	0.0222
Seed 7	0.0667

The model is the same in every row of the table. Only the split changes. Seed 42 reports a perfect classifier, and seed 7 reports one that is wrong on one case in fifteen. Both are "the" holdout estimate. With the training set cut to 30 rows the spread widens to mean 0.0572, standard deviation 0.0296, and worst split 0.4083. A holdout score is therefore reported with its split, seed and test size. A difference of 0.02 between two small holdout samples does not by itself show that one workflow is better.

Random subsampling Repeat the random split r times. Each repetition draws a fresh training sample without replacement and tests on the remainder, giving $Score_1, \ldots, Score_r$. Report their average and describe the split variability. Test sets from different repetitions may overlap, and some rows may never be tested.

Setup code Every listing in this handout runs, top to bottom in a fresh session, after this setup. R ships `iris`; Python loads it from scikit-learn. `knn` is the 3-nearest-neighbour classifier used throughout.

R

```r
library(caret)    # knn3, train, createFolds
library(rsample)  # mc_cv
library(boot)     # boot
```

Python

```python
import numpy as np
from sklearn.datasets import load_iris
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import (
    GridSearchCV,
    KFold,
    ShuffleSplit,
    StratifiedKFold,
    TimeSeriesSplit,
    cross_val_score,
    cross_validate,
    train_test_split,
)
from sklearn.neighbors import (
    KNeighborsClassifier,
)

X, y = load_iris(return_X_y=True)
knn = KNeighborsClassifier(3)
```

Holdout and random subsampling in R and Python

R

```r
set.seed(3141)
i  <- sample(nrow(iris), 0.7 * nrow(iris))
tr <- iris[i, ]; te <- iris[-i, ]
m  <- knn3(Species ~ ., tr, k = 3)
mean(predict(m, te, type = "class") !=
     te$Species)
# random subsampling: 10 repeated splits
splits <- mc_cv(iris, prop = 0.7, times = 10)
err <- vapply(splits$splits, function(s) {
  m  <- knn3(Species ~ ., analysis(s), k = 3)
  te <- assessment(s)
  mean(predict(m, te, type = "class") !=
       te$Species)
}, numeric(1))
c(mean = mean(err),
  naive_se = sd(err) / sqrt(length(err)))
```

Python

```python
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.30, random_state=3141)
print(1 - knn.fit(Xtr, ytr).score(Xte, yte))
# random subsampling: 10 repeated splits
cv = ShuffleSplit(n_splits=10, test_size=0.30,
                  random_state=3141)
err = 1 - cross_val_score(knn, X, y, cv=cv)
print(err.mean(), err.std(ddof=1) / np.sqrt(10))
```

With seed 3141 the single holdout reports 0.0667 in Python and 0.1111 in R, and ten repeated splits average 0.0444 and 0.0378, a reminder that one split is one draw. `ShuffleSplit` is scikit-learn's random subsampling, repeated independent splits whose test sets may overlap [11]. It is not K-fold, even though both are passed to the same `cv=` argument.

4 The bootstrap and the .632 estimate

What it is for The bootstrap [5] trains on a sample as large as the full data set while still leaving unseen cases to test on. It is the usual choice for small data sets.

Procedure

1. Train a model on a random sample of size N, drawn **with replacement** from the original data set of size N. Some cases appear more than once in the training set.
2. Test the model on the cases that did *not* appear in the training set (the out-of-bag cases).
3. Repeat many times, typically around 200.

Figure 1 shows three resamples of a nine-case sample.

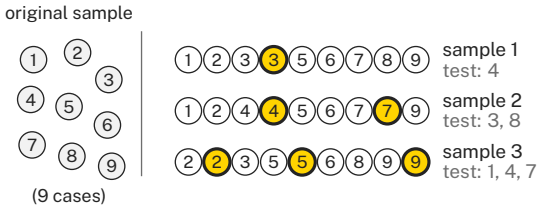


Figure 1. Three bootstrap resamples. Gold marks the extra copy of a case drawn more than once. Each extra copy takes the place of a case that never enters training, and those cases are the resample's test set. After Galdi and Tagliaferri [8].

Why 63.2 per cent One draw misses a given case with probability $1 - 1/N$, and all N draws miss it with probability $(1 - 1/N)^N \to e^{-1} = 0.3679$. On average, therefore, about **63.2 per cent** of the distinct cases enter the training set and the remaining **36.8 per cent** are available for testing. Over 200 resamples of `iris` with the Python generator below, `default_rng(3141)`, the observed mean is 63.57 per cent, which is the limit $1 - e^{-1} = 0.6321$ plus finite-N and sampling variation.

The .632 estimate The average out-of-bag score over the repetitions is ϵ_0. With ϵ_r the resubstitution estimate, the **.632 bootstrap estimate** [6] is

$$0.368 \times \epsilon_r + 0.632 \times \epsilon_0.$$

ϵ_0 is computed on unseen cases, but each model saw only about 63 per cent of the distinct cases, so ϵ_0 is pessimistic. ϵ_r is optimistic for the reasons in Section 1. The weights are the same proportions the resampling produces, and the result lies between the two.

Worked example 3-NN on `iris`, 200 bootstrap repetitions:

$$\epsilon_0 \approx 0.04607, \qquad \epsilon_r = 0.0400,$$

$$0.368(0.0400) + 0.632(0.04607) \approx 0.01472 + 0.02912 \approx \mathbf{0.0438}.$$

Reporting ϵ_r alone would claim 0.0400 and ϵ_0 alone 0.0461. The .632 estimate is 0.0438. For comparison, stratified 10-fold CV on the same data and model gives 0.0467.

The bootstrap in R and Python

R

```r
oob_err <- function(d, idx) {
  tr <- d[idx, ]; te <- d[-unique(idx), ]
  m  <- knn3(Species ~ ., tr, k = 3)
  mean(predict(m, te, type = "class") !=
       te$Species)
}
set.seed(3141)
b  <- boot(iris, oob_err, R = 200)
e0 <- mean(b$t)
m  <- knn3(Species ~ ., iris, k = 3)
er <- mean(predict(m, iris, type = "class") !=
           iris$Species)
0.368 * er + 0.632 * e0           # 0.0452
trainControl()$method   # "boot" (number = 25)
```

Python

```python
rng = np.random.default_rng(3141)
n, errs = len(y), []
for _ in range(200):
    idx = rng.integers(0, n, n)
    oob = np.setdiff1d(np.arange(n), idx)
    m = KNeighborsClassifier(3).fit(
        X[idx], y[idx])
    errs.append(1 - m.score(X[oob], y[oob]))
e0 = np.mean(errs)                    # 0.0461
er = 1 - knn.fit(X, y).score(X, y)    # 0.0400
print(0.368 * er + 0.632 * e0)        # 0.0438
```

Differences to know

- `trainControl()$method` defaults to `"boot"`, so caret resamples with the *bootstrap*, 25 repetitions [10]. A user of `caret::train` who never sets `method` is bootstrapping.
- scikit-learn has no bootstrap splitter in `model_selection`. The explicit loop above is how it is written. In R, `boot::boot` runs the resampling loop [3].
- R and Python draw different resamples from the same seed, so the R value of ϵ_0 differs from 0.0461 and its .632 estimate is 0.0452.

5 K-fold cross-validation

What it is for Cross-validation tests every row exactly once, so no row is wasted and no test sets overlap.

Procedure

1. Randomly permute the rows.
2. Divide them into K blocks of the same size, called **folds**.
3. For each fold, test a model trained on the remaining $K - 1$ folds.

This gives K scores, one per fold. Report their mean and spread as in Section 2. Unlike random subsampling and the bootstrap, **there is no overlap between test sets**. Every row is tested exactly once and used for training exactly $K - 1$ times. Figure 2 shows $K = 5$.

Choosing K Common values are 5, 10 and 20; Kohavi [9] recommends stratified 10-fold CV as the default. With $K = N$, where N is the number of observations, the method is **leave-one-out cross-validation** (LOOCV). Each model is trained on all rows but one and tested on that row. The whole process can be repeated, for example 10×10-fold CV, which averages over the random permutation. Table 3 gives the trade-offs.

Worked example 3-NN on `iris`, mean $\pm$ naive SE. Stratified 5-fold gives 0.0333 ± 0.0149, stratified 10-fold gives 0.0467 ± 0.0174, and LOOCV gives 0.0400. The three differ by about one case out of

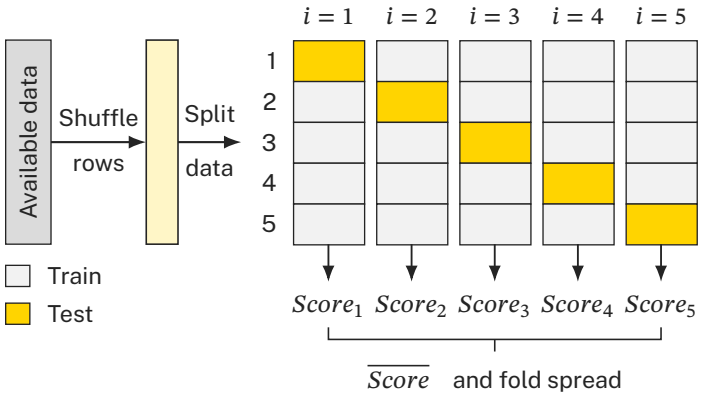


Figure 2. Five-fold cross-validation. The rows are shuffled and split into five folds. In split i the gold fold is the test set and the other four train the model, giving one score per split.

Table 3. Trade-offs of small, large and leave-one-out fold counts

K small (5)	Each training set is only 80 per cent of the data, so the fitted models are slightly weaker than the final model and the estimate is slightly pessimistic. Cheap: five fits. Fold scores are relatively stable because each test fold is large.
K large (20)	Each model sees 95 per cent of the data, so the estimate is closer to the final model's performance. Expensive: twenty fits. The training sets overlap heavily, so the fold scores are strongly correlated and the standard error is understated.
$K = N$ (LOOCV)	Nearly unbiased for the final model's error, but there is no randomness left to average over (LOOCV is deterministic) and it costs N fits. Each test set has one row, so individual fold scores are 0 or 1.

150, which is the resolution of a 150-row data set, and the reason to repeat the whole process on small data.

Cross-validation in R and Python

R

```r
set.seed(3141)
ctrl <- trainControl(method = "cv", number = 5)
fit  <- train(Species ~ ., data = iris,
              method = "knn",
              tuneGrid = data.frame(k = 3),
              trControl = ctrl)
1 - fit$results$Accuracy          # 0.0400
```

Python

```python
cv = StratifiedKFold(n_splits=5, shuffle=True,
                     random_state=3141)
err = 1 - cross_val_score(knn, X, y, cv=cv)
print(err.mean())                      # 0.0333
# the training score beside the held-out one
res = cross_validate(knn, X, y, cv=cv,
                     return_train_score=True)
print(1 - res["train_score"].mean(),  # 0.0367
      1 - res["test_score"].mean())   # 0.0333
```

`cross_validate` with `return_train_score=True` returns the resubstitution score beside the held-out score for every fold. For 3-NN the two errors are close, 0.0367 and 0.0333. For 1-NN the resubstitution error is zero, which is the gap Table 1 measures.

6 Stratified folds and the sorted-file trap

What it is for Random folds can be unrepresentative, and folds cut from a sorted file always are. Stratifying and shuffling fix the two cases.

Stratified K-fold CV In regular CV a fold may not represent the available data. **Stratified CV** uses stratified instead of random sampling so that the mean response value is roughly the same in every fold. In binary classification this keeps roughly the same relative frequency of the two classes in every fold. The cost is that stratifying on the response compromises the "purity" of the random fold allocation. The folds are no longer a simple random partition, and theory that assumes one no longer applies exactly.

Worked example: an imbalanced class Take 1000 rows with 12 positives and unstratified 10-fold CV, so each fold holds 100 rows. The probability that a given fold contains no positive is

$$\binom{988}{100} \Big/ \binom{1000}{100} = 0.2803,$$

so on average 2.8 of the 10 folds have no positive at all. On those folds recall and F_1 are undefined. Stratification places one or two positives in every fold.

The sorted-file trap `iris` is stored sorted by species, rows 1–50 setosa, 51–100 versicolor and 101–150 virginica. Cut three folds without shuffling and each fold *is* one species. The held-out species never appears in its own training set, so it can never be predicted.

R

```r
# accuracy of 3-NN, one value per held-out fold
fold_acc <- function(folds) {
  sapply(folds, function(te) {
    tr <- droplevels(iris[-te, ]) # unseen class
    m <- knn3(Species ~ ., tr, k = 3)
    p <- predict(m, iris[te, ], type = "class")
    mean(as.character(p) == iris$Species[te])
  })
}
ord <- rep(1:3, each = 50)        # file order
fold_acc(split(seq_len(nrow(iris)), ord))
# 0 0 0    <-- fold i IS species i, no warning
set.seed(3141)
f <- createFolds(iris$Species, k = 3)
mean(fold_acc(f))
# 0.947  <-- createFolds permutes AND stratifies
```

Python

```python
print(cross_val_score(knn, X, y, cv=KFold(3)))
# [0. 0. 0.]   <-- shuffle=False is the default
cv = KFold(3, shuffle=True, random_state=3141)
print(cross_val_score(knn, X, y, cv=cv).mean())
# 0.9667
```

The two listings build the same split. `KFold(3)` with `shuffle=False` cuts rows 1–50, 51–100 and 101–150, which is what `rep(1:3, each = 50)` builds in R. The zero is forced by the data, not by the seed. In R the failure has to be built by hand, because `createFolds` permutes and stratifies before it cuts. In Python an integer `cv=3` stratifies a classification target, and the failure appears only when plain `KFold(3)` is passed explicitly.

Zero accuracy in every fold is easy to notice. The five-fold version in Table 4 is not. The unshuffled folds lose six percentage points of accuracy with no error and no warning. Any file sorted by class, date, customer ID or collection order has this property.

Table 4. Accuracy under three five-fold splitting strategies

Splitter, five folds	Accuracy
`KFold(5)`, unshuffled	0.9067
`KFold(5, shuffle=True)`	0.9600
`StratifiedKFold(5, shuffle=True)`	0.9667

7 Library defaults that change the answer

What it is for Four library defaults decide the estimate unless you set them, and the seed decides whether anyone can repeat it.

Differences to know

Which resampler you get by default.
In R, `caret::train` resamples with the **bootstrap**, 25 repetitions; cross-validation happens only if you write `trainControl(method = "cv")`. In Python, `cross_val_score` uses **5-fold**, and only 5-fold.

Whether rows are shuffled.
In R, `createFolds` randomizes fold assignment. In Python, both `KFold` and `StratifiedKFold` default to `shuffle=False`. Integer `cv` stratifies a classification target; explicitly passing plain `KFold` on a class-sorted file can omit whole classes from training.

Whether folds are stratified.
In R, `createFolds` stratifies on `y` by default, and `trainControl(method = "cv")` builds its folds with it. To use any other assignment, pass the fold list through `trainControl(index = ...)`, which uses exactly the folds you give it. In Python, the folds are stratified for a **classifier** and plain `KFold` for a **regressor**. Passing `cv=5` and passing `cv=KFold(5)` are not the same call, even though both look like "five-fold".

Sign of the score.
R reports error metrics as errors. Smaller is better. scikit-learn's scorers are written so that **larger is always better**, so error metrics are negated. `"neg_mean_squared_error"` returns $-MSE$. Flip the sign before reporting.

Reproducibility.
R: `set.seed()` before the call. Python: `random_state=` on the splitter *and* on any stochastic estimator.

Write the splitter out in full. `cv=5` means whatever scikit-learn infers, which depends on whether the estimator is a classifier. `StratifiedKFold(5, shuffle=True, random_state=3141)` means the same thing every time it runs.

8 Choosing a method by data set size

What it is for The four methods trade compute against how much each row is reused. Data set size decides which trade is worth making.

Rule of thumb Table 5 recommends a method by data set size.

Table 5. Resampling method recommendations by data set size

Data set	Recommended method	Why
Small (< 500 cases)	Leave-one-out CV, or the bootstrap with around 200 repetitions; the bootstrap is usually preferred	Every case is scarce. LOOCV is nearly unbiased but unstable, because its N training sets are nearly identical; the bootstrap accepts a little pessimism for a much more stable estimate
Medium (a few thousand rows)	10-fold cross-validation, often repeated several times to reduce sensitivity to one partition	Ten folds balance the pessimism of small K against the cost and correlated folds of large K
Large	Random subsampling; if the data are too large or the model too costly to refit, holdout	A single split already has enough test rows to be stable, and refitting dominates the cost

Worked example With 300 rows every row matters and 200 model fits are cheap, so the bootstrap or LOOCV is appropriate. With 3 million rows a single 70/30 split leaves 900 000 test rows, enough for a stable estimate, and 200 fits may take a day of compute, so holdout is appropriate.

9 Model selection: internal and external evaluation

What it is for Choosing a model, or its hyper-parameters, is itself a use of the data. If the test set makes the choice, it has been used during model development, the Golden Rule is broken, and the reported score is optimistic for the same reason a resubstitution score is.

Rule

1. Use an **internal** evaluation method to select the best model and/or its hyper-parameters.
2. Use an **external** evaluation method to estimate performance on unseen data. This is the estimate presented to the end-user.

Nested evaluation keeps model selection inside each outer training split. The internal and external methods need not be the same.

Nested holdout The data are split into training and test. The training part is split again into training and validation. Parameters are chosen on the inner split, and the outer test block is used once, for the final evaluation (Figure 3). Every choice, including features, hyper-parameters, thresholds and which model wins, is made without the test block.

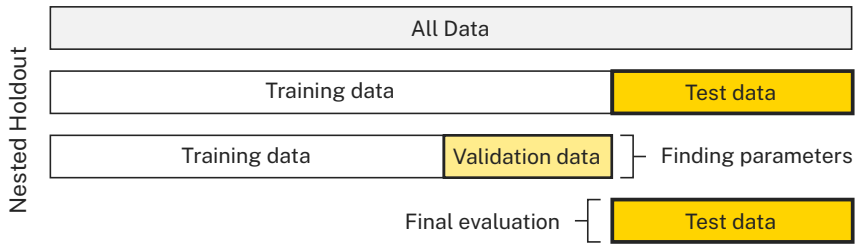


Figure 3. Nested holdout. Parameters are found on the inner training and validation split. The test block is set aside once and used once, for the final evaluation.

Grid search inside cross-validation Selecting the best hyper-parameters is **hyper-parameter tuning**. Testing every combination of a manually selected subset of hyper-parameters is **grid search**. The second design replaces the inner holdout with 5-fold CV. Internal evaluation (also called *model validation*) is 5-fold CV on the training portion, and external evaluation is the outer holdout.

Worked example Tune $k \in \{1, 3, 5, 9\}$ for k-NN on a stratified 70/30 split of `iris`. The code below chooses k by 5-fold CV on the 105 training rows and scores the chosen model once on the 45 test rows.

Nested evaluation in R and Python

R

```r
set.seed(3141)
i  <- createDataPartition(iris$Species,
        p = 0.7, list = FALSE)   # stratified
tr <- iris[i, ]; te <- iris[-i, ]
# INTERNAL: 5-fold CV over the grid, on tr only
grid <- data.frame(k = c(1, 3, 5, 9))
ctrl <- trainControl(method = "cv", number = 5)
fit  <- train(Species ~ ., data = tr,
              method = "knn", tuneGrid = grid,
              trControl = ctrl)
fit$bestTune                       # k = 9
# EXTERNAL: te is touched exactly once
mean(predict(fit, te) != te$Species)  # 0
```

Python

```python
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.30, random_state=3141,
    stratify=y)
# INTERNAL: 5-fold CV over the grid, on Xtr only
inner = StratifiedKFold(5, shuffle=True,
                        random_state=3141)
gs = GridSearchCV(KNeighborsClassifier(),
                  {"n_neighbors": [1, 3, 5, 9]},
                  cv=inner)
gs.fit(Xtr, ytr)
print(gs.best_params_)    # {'n_neighbors': 1}
# EXTERNAL: Xte is touched exactly once
print(1 - gs.score(Xte, yte))     # 0.0
```

Python's search picks $k = 1$ and R's picks $k = 9$, because the two languages cut different inner folds; both chosen models classify all 45 test rows correctly.

Fully nested cross-validation Replace the outer holdout with an outer resampling loop.

R

```r
set.seed(3141)
inner <- trainControl(method = "cv", number = 5)
outer <- createFolds(iris$Species, k = 5,
                     returnTrain = TRUE)
mean(sapply(outer, function(tr) {
  fit <- train(Species ~ ., data = iris[tr, ],
               method = "knn", tuneGrid = grid,
               trControl = inner)  # INSIDE
  p <- predict(fit, iris[-tr, ])   # OUTSIDE
  mean(p != iris$Species[-tr])
}))                                # 0.0200
```

Python

```python
outer = StratifiedKFold(5, shuffle=True,
                        random_state=3141)
res = cross_validate(gs, X, y, cv=outer)
print(1 - res["test_score"].mean())   # 0.0467
```

In caret the inner loop is `trainControl` and the outer loop is written by hand. In scikit-learn a search object is itself an estimator, so it can be passed to an outer resampler as if it were a model. The outer estimates are 0.0467 in Python and 0.0200 in R.

Fully nested CV costs $K_{\text{outer}} \times K_{\text{inner}} \times |\text{grid}|$ fits. Above, that is $5 \times 5 \times 4 = 100$ fits (plus one refit per outer fold). Use it when the data set is too small for a single outer holdout to be stable.

The number you must not report

`gs.best_score_` (and `fit$results` in caret) is the *internal* score of the winning configuration. It is the maximum over the grid of a noisy quantity, so it is biased upward by the selection itself. It is a selection statistic, never the reported performance.

10 Performance estimation for time series

What it is for Every method so far shuffles or resamples rows at random. Time-ordered data forbid that, and this section gives the replacements.

Why random resampling fails Common evaluation methods sample rows at random and so assume no order, and no dependence, between observations. A time series has an inherent order and often dependence as well. If a shuffled test fold sits in the middle of the series, its training set contains later observations, and the model is scored on a question whose answer is partly in its training data. The resulting score can look excellent and is worthless.

Guidelines

- Do not ignore temporal order.
- Do not evaluate by training on "future" data and testing on "past" data.

Simple train-test windows Divide the data into past data (observations up to a time t) and "future" data (observations after t). Figure 4 draws the three schemes.

- **Fixed window.** One model is fitted on the training data and applied to the whole test period.
- **Growing window.** Every w_{test} test cases, a new model is fitted on all data up to that point. The training window grows.
- **Sliding window.** Every w_{test} test cases, a new model is fitted on the previous w_{train} observations. The training window has fixed length and moves forward, discarding the oldest data.

A growing window assumes the relationship is stable, so more history helps. A sliding window assumes the relationship drifts, so old data mislead.

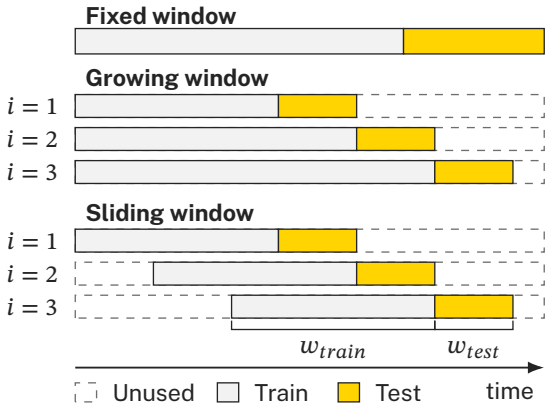


Figure 4. Fixed, growing and sliding train-test windows. Training always precedes testing.

Monte Carlo estimates

1. Select a training window size w_{train} and a testing window size w_{test}.
2. Randomly generate r points $s_1, \dots, s_r$ within the interval $[w_{train}, \dots, (n - w_{test})]$.
3. For each point, train on the previous w_{train} observations and test on the next w_{test} observations.

The interval leaves room for both windows. A point closer than w_{train} to the start has no room to train, and one closer than w_{test} to the end has no room to test. To tune hyper-parameters, place an internal train-test window inside each training window, which is the nesting of Section 9 applied along the time axis. Cerqueira, Torgo, and Mozetič [4] compare these estimators empirically across many real series.

Worked example The listings below simulate a series of $n = 200$ points. In the R listing $w_{train} = n/2 = 100$ and $w_{test} = n/4 = 50$, so the $r = 10$ split points are drawn from $\{100, \dots, 150\}$, and each test window is the 50 observations after its split point.

Time-ordered estimation in R and Python

R

```r
set.seed(3141)                    # a series
n <- 200; x <- cumsum(rnorm(n))
ts_df <- data.frame(x, y = 0.8 * x + rnorm(n))
w_train <- n / 2; w_test <- n / 4   # 100, 50
s_pts <- sample(w_train:(n - w_test), 10)
mse <- vapply(s_pts, function(s) {
  tr <- (s - w_train + 1):s
  te <- (s + 1):(s + w_test)
  fit <- lm(y ~ x, ts_df[tr, ])
  e <- predict(fit, ts_df[te, ]) - ts_df$y[te]
  mean(e^2)
}, numeric(1))
mean(mse)                         # 1.256
# growing window inside train()
grow <- trainControl(method = "timeslice",
  initialWindow = 100, horizon = 10,
  fixedWindow = FALSE)
```

Python

```python
rng = np.random.default_rng(3141)  # a series
x = np.cumsum(rng.normal(size=200))
Xts = x.reshape(-1, 1)
yts = 0.8 * x + rng.normal(size=200)
# growing window: train always precedes test
cv = TimeSeriesSplit(n_splits=5, test_size=10)
for tr, te in cv.split(Xts):
    assert tr.max() < te.min()
# sliding window: cap the training length
cv = TimeSeriesSplit(n_splits=5, test_size=10,
                     max_train_size=100)
scores = cross_val_score(LinearRegression(),
    Xts, yts, cv=cv,
    scoring="neg_mean_squared_error")
print(-scores.mean())   # 1.207, sign flipped
```

Differences to know

- `TimeSeriesSplit` never shuffles. Here the unshuffled behaviour that caused the sorted-file trap in Section 6 is the correct one.
- `max_train_size` turns the growing window into a sliding one. In caret the same switch is `fixedWindow = TRUE`.

11 Common misconceptions

1. **"My training accuracy is 100 per cent, so the model is excellent."** A 1-NN classifier achieves that on every data set with no duplicate rows and no contradictory labels. Training accuracy measures fit, not performance.
2. **"The test set told me model B is better, so I will use model B and report its test score."** Once the test set chose between A and B it became part of model development, and its score became optimistic. Choose on an internal estimate and report the external one.

3. **"Cross-validation gives the performance of the model I will deploy."** It gives the expected performance of the *procedure*, meaning this algorithm, with these settings, fitted to $N(1 - 1/K)$ rows drawn this way. The model you deploy is usually refitted on all N rows and is slightly better than the estimate suggests [1].
4. **"More folds is always better."** More folds reduce the pessimism of the estimate but increase the correlation between fold scores and the cost. Repeated 10-fold CV balances the two.
5. **"I set `shuffle=True`, so I do not need stratification."** Shuffling fixes the ordering problem. It does not guarantee that a rare class appears in every fold. Only stratification does that.
6. **"Bootstrap and random subsampling are the same thing."** Random subsampling draws *without* replacement, so no case appears twice in a training set. The bootstrap draws *with* replacement, which is what produces the 63.2 / 36.8 split and makes the .632 correction necessary.
7. **"The `neg_mean_squared_error` came out negative, so my model has negative error."** scikit-learn negates error metrics so that larger is always better. Petal width regressed on petal length (`iris`, shuffled 5-fold, seed 3141) scores −0.0442, so the MSE is 0.0442 cm^2.
8. **"I will just use cross-validation on my time series."** Random folds train on the future. Use a fixed, growing, sliding or Monte Carlo window.

12 Summary

- **Holdout.** Train on a random sample of the data (without replacement) and test on the remaining observations.
- **Random subsampling.** Repeat holdout several times, with different random samples each time.
- **Bootstrap.** Train on a random sample drawn *with* replacement and test on the remaining cases, usually repeated around 200 times for small data sets. About 63.2 per cent of the distinct cases land in the training sample; the .632 estimate is $0.368 \times \epsilon_r + 0.632 \times \epsilon_0$.
- **K-fold cross-validation.** Randomly sample the data into K folds and iteratively use each fold as a test set and the others as the training set. Leave-one-out CV if $K = N$.
- **Stratified K-fold CV.** Use stratified sampling to create the splits and keep the mean response value roughly the same across folds.
- **Nest** evaluation methods to select the model and/or the model hyper-parameters; report the outer estimate.
- **Time series.** The common model evaluation methods cannot be used. Use alternatives such as a sliding window.

Reference numbers used in these notes

Every number quoted above was computed on `iris` ($N = 150$, three balanced classes of 50) with a 3-nearest-neighbour classifier unless stated otherwise, seed 3141. Table 6 collects them.

Table 6. Reference values for the worked cross-validation examples

Quantity	Value
Resubstitution error, $k = 1$ / $k = 3$	0.0000 / 0.0400
Stratified 10-fold CV error, $k = 1$	0.0400
Holdout 70/30, 1000 splits	mean 0.0393, sd 0.0232, range 0.0000–0.1333
Holdout with 30 training rows	mean 0.0572, sd 0.0296, worst 0.4083
Stratified 5-fold CV (mean ± naive SE)	0.0333 ± 0.0149
Stratified 10-fold CV (mean ± naive SE)	0.0467 ± 0.0174
LOOCV	0.0400
Bootstrap, 200 reps: distinct cases	63.57 per cent observed, $1 - e^{-1} = 0.6321$
Bootstrap ϵ_0 / ϵ_r / .632	0.0461 / 0.0400 / 0.0438
`KFold(3)` unshuffled on sorted `iris`	accuracy 0.000 in all three folds
`KFold(5)` unshuffled / shuffled / stratified	0.9067 / 0.9600 / 0.9667

References

[1] S. Bates, T. Hastie, and R. Tibshirani. “Cross-validation: what does it estimate and how well does it do it?” In: *Journal of the American Statistical Association* 119.546 (2024), pp. 1434–1445.

[2] Y. Bengio and Y. Grandvalet. “No unbiased estimator of the variance of K-fold cross-validation”. In: *Journal of Machine Learning Research* 5 (2004), pp. 1089–1105.

[3] A. Canty and B. Ripley. *boot: Bootstrap Functions*. CRAN.

[4] V. Cerqueira, L. Torgo, and I. Mozetič. “Evaluating time series forecasting models: an empirical study on performance estimation methods”. In: *Machine Learning* 109.11 (2020), pp. 1997–2028.

[5] B. Efron. “Bootstrap methods: another look at the jackknife”. In: *The Annals of Statistics* 7.1 (1979), pp. 1–26.

[6] B. Efron. “Estimating the error rate of a prediction rule: improvement on cross-validation”. In: *Journal of the American Statistical Association* 78.382 (1983), pp. 316–331.

[7] R. A. Fisher. “The use of multiple measurements in taxonomic problems”. In: *Annals of Eugenics* 7.2 (1936), pp. 179–188.

[8] P. Galdi and R. Tagliaferri. “Data mining: accuracy and error measures for classification and prediction”. In: *Encyclopedia of Bioinformatics and Computational Biology*. Elsevier, 2019, pp. 431–436.

[9] R. Kohavi. “A study of cross-validation and bootstrap for accuracy estimation and model selection”. In: *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI-95)*. Vol. 2. 1995, pp. 1137–1143.

[10] M. Kuhn. *caret: Classification and Regression Training*. Version 7.0-1. Package documentation for model training, resampling, tuning and evaluation. CRAN. 2024.

[11] scikit-learn developers. *Cross-validation: Evaluating Estimator Performance*. User guide, with `KFold`, `StratifiedKFold`, `ShuffleSplit`, `TimeSeriesSplit`, `cross_val_score`, `cross_validate` and `GridSearchCV`. URL: `https://scikit-learn.org/stable/modules/cross_validation.html`.